X5 TECH MEETUP · INTERNAL · EP02 · FOUNDATION

AICreator_

Платформа детерминированной генерации кода из Proto, OpenAPI, AsyncAPI и PUML. От CTO-демо на трёх языках — к пилоту на Go.

```
$ aicreator generate --function f1 --language go
```

X5 Tech

AGENDA · 20 MIN

О чём поговорим

01 Идея продукта и зачем он нужен 3 min

02 4 функции генерации 3 min

03 Два эпика: Demo → Foundation 3 min

04 Как устроен пайплайн 3 min

05 Живой CLI: что видит разработчик 3 min

06 Как я это разрабатывал 2 min

07 Дорожная карта · Q&A 3 min

“

Бойлерплейт жрёт время. AICreator ест его за нас — детерминированно.

— ТЕЗИС ДОКЛАДА

PROBLEM → SOLUTION

От «пишем DTO руками» к «генерим из спеки»

СЕЙЧАС

Копипаст, который дрейфует

- ✗ Команды на Kotlin / Go / Python пишут DTO и клиенты с нуля
- ✗ Устные контракты → расхождение спеки и кода за пару спринтов
- ✗ Новый сервис — дни на onboarding и обвязку
- ✗ Стандарты хедеров, неймингов живут в головах

→

С AICREATOR

Спека — источник правды

- ✓ Один CLI:

```
aicreator
generate
```

- ✓ Proto · OpenAPI — на входе. Docker — единственное требование
- ✓ Детерминизм: SHA-256 хэш входа = идентичный выход
- ✓ Fail fast: ошибка в спеке — мгновенная остановка

SCOPE · 4 ФУНКЦИИ ГЕНЕРАЦИИ

Что умеет платформа

F1 · ДЕТЕРМ.

Дата-модели

.proto → структуры

gRPC-stubs и message-типы. Контракт — основа микросервиса.

buf 1.50.0 · protoc 29.3

F2 · ДЕТЕРМ.

REST-сервер

openapi.yaml → handlers

Роутеры, контроллеры, модели. Команде — только бизнес-логика.

openapi-generator 7.12.0 · go-server

F3 · AI · Q4

Сервисы из PUML

.puml → скелет

Интерфейсы из class- и sequence-диаграмм. Только скелет, не логика.

litellm → Qwen 3 235B / GLM 4.7

F4 · ДЕТЕРМ.

Клиенты

openapi.yaml → client

Типизированный HTTP-клиент. Retry, таймауты, pydantic-валидация.

openapi-generator 7.12.0 · client

ПУТЬ ПРОДУКТА · ДВА ЭПИКА

Сначала — доказать идею. Потом — дать в руки команде.

EP01 · DEMO · DONE

Проверка идеи для CTO

Audience: CTO · мульти-язык · доказательство концепции

Docker-пайплайн на одной команде: генерация → форматирование → валидация. Три волны, девять контейнеров, три языка параллельно.

- F1 · F2 · F4 на Kotlin, Go, Python
- buf + openapi-generator, всё pinned
- Три акта: happy path · volume swap · fail fast

307	9	3
ФАЙЛОВ / ПРОГОН	DOCKER-СЕРВИСОВ	ЯЗЫКА

EP02 · FOUNDATION · DONE

Продукт для пилотной команды

Audience: одна команда X5 Tech · Go-стек · продукт

CLI + FastAPI, PostgreSQL с input-hash gate, Docker-образ с Go-тулчейном. Фокус на одном стеке — Go, все четыре реализованные функции для него.

- CLI (typer) · API (FastAPI) · Orchestrator (Registry)
- F1 · F2 · F4 для Go, детерминизм в БД
- PostProcessor: gofmt → golangci-lint → go build

~104	1	9
GO-ФАЙЛОВ / СПЕКА	КОМАНДА В ПИЛОТЕ	ПРИНЦИПОВ

LIVE CLI · EP02

aicreator generate

~/x5/aicreator — aicreator · zsh

```
$ which go # проверяем: на хосте только docker
go not found
$ docker compose --profile prod up -d # поднимаем платформу
✓ postgres · healthy
✓ api-prod · Application startup complete

$ aicreator generate -f f2 -s ./specs/logistics -o ./out -l go
→ Uploading spec logistics.yaml (sha256: 7a3c…b9)
→ Orchestrator: openapi → go → f2 [OpenAPIServerGenerator]
→ openapi-generator 7.12.0 · go-server ✓ 8.2s
→ gofmt → golangci-lint → go build ✓ 3.1s

✓ Generated 38 Go files → ./out/go
✓ generation_id: 5f4c-02d9-… · input_hash в БД

$ aicreator generate -f f2 ... # повторный запуск
✓ diff -r out/ out-2/  (0 bytes)  — детерминизм
```

ПОД КАПОТОМ · ИНСТРУМЕНТЫ ГЕНЕРАЦИИ

Что именно крутится внутри контейнера

F1 · PROTO → GO

buf + protoc

buf 1.50.0 · protoc 29.3

buf.yaml для валидации модуля, buf.gen.go.yaml для плагинов. protoc-gen-go и protoc-gen-go-grpc выдают structs и grpc-stubs.

buf 13k ⭐ · BSR registry

F2 · OPENAPI → SERVER

openapi-generator

openapi-gen 7.12 · go-server

Java-JAR под JRE 21, mustache-шаблоны для корпоративного хедера. Роутер на gorilla/mux, interface-based controllers.

23k ⭐ · 50+ языков из коробки

F4 · OPENAPI → CLIENT

openapi-generator

openapi-gen 7.12 · go

Тот же JAR, другой generator-name. Типизированный клиент, retry/timeout конфигурируются через Config-struct.

Совместимость server ↔ client из одной спеки

POST · ФОРМАТ

gofmt

Go 1.23 → canonical

Первый шаг PostProcessor. Приводит весь сген.код к каноническому виду — иначе детерминизм сломан.

встроен в Go toolchain

POST · ЛИНТ

golangci-lint

1.63.4 → report

Мета-линтер: errcheck, govet, staticcheck. Конфиг под корпоративные правила. Ошибка — fail fast.

16k ⭐ · индустриальный стандарт

POST · BUILD

go build

go 1.23 → ✓ compile

Финальный gate. Если сген.код не компилируется — генерация неуспешна. Команда получает не мусор, а готовый артефакт.

gate перед ZIP-ответом

PIPELINE · EP02

Как устроен пайплайн

INTERFACE

CLI · typer
aicreator generate / status / health

API · FastAPI
POST /api/v1/generate · multipart

↓

CORE

Orchestrator · Registry
routing по (spec_type, language, function) → конкретный генератор

↓

GENERATORS

BufGoGenerator · F1
proto → go, buf 1.50.0

OpenAPIServerGen · F2
yaml → go server, gen 7.12

OpenAPIClientGen · F4
yaml → go client, gen 7.12

↓

POST · STORE

PostProcessor
gofmt → golangci-lint → go build · fail-fast

PostgreSQL
input_hash · status · duration · ZIP стримится клиенту

CONSTITUTION · 9 ПРИНЦИПОВ

С чем не торгуемся — всё в коде

01 Determinism first
Одинаковый вход → одинаковый выход. SHA-256 в БД — gate.

02 Containerized platform
На хосте только Docker. Ни Go, ни Java, ни Python ставить не надо.

03 Fail fast, fail loud
Любая ошибка останавливает пайплайн с понятным сообщением.

04 Spec agnostic
Новый тип спеки = новый класс + запись в Registry. Ядро не трогаем.

05 Pinned versions
Явные теги у всех образов и инструментов. Никаких `:latest`.

06 Minimal infrastructure
Только PostgreSQL. Никаких Redis, Kafka, очередей.

07 Template-driven
Корпоративные хедеры, нейминг — в шаблонах, не в коде генератора.

08 OSS 700+ ⭐
FastAPI · buf · openapi-generator — всё проверено сообществом.

09 Go first · multi-lang by design
Сейчас — только Go. Архитектура не блокирует Kotlin и Python.

PROCESS · КАК Я ЭТО СОБИРАЛ

От интервью с CTO до pilot-ready в четыре волны

Step 1 · research

Research & ADR

Интервью с CTO, решения зафиксированы в ADR. OSS-gate 700+ ⭐, выбор стека: Python + FastAPI + typer.

- master-plan.md
- research.md
- ADR-001 … ADR-006

Step 2 · demo

ep01 — доказать идею

Docker Compose с depends_on. Три языка параллельно. Три акта для CTO: happy path, volume swap, fail fast.

- 9 контейнеров
- pinned images
- live на Mac

Step 3 · platform

ep02 — 4 волны, 15 тасок

Skeleton → Core → API → CLI. Каждая волна — работающий инкремент. PostgreSQL + Alembic с первого коммита.

- T01–T04 skeleton
- T05–T08 core
- T09–T15 API + CLI

Step 4 · ship

Prod-образ & E2E

Multi-stage Dockerfile: Python + Go + JRE + buf в одном образе. E2E: CLI → API → Generator → файлы на диске.

- docker compose up
- aicreator health
- ready for pilot

EP01 · EP02 · СРАВНЕНИЕ

Что изменилось между эпиками

	EP01 · DEMO	EP02 · FOUNDATION
Языки на выходе	Kotlin · Go · Python	Go · остальные архитектурно готовы
Запуск	`docker compose up`	`aicreator generate -f f2 …`
Оркестрация	Docker Compose · depends_on	Registry · Strategy · subprocess
Хранение результата	Файлы в ./generated	ZIP stream + input_hash в Postgres
Повторяемость	Скрипты	SHA-256 gate, подтверждённый в БД

ROADMAP · 2026

Куда едем дальше

Q1 · DONE

ep01 — Demo

Docker-пайплайн, три языка, три акта.

- F1 · F2 · F4
- Pinned images

Q2 · DONE

ep02 — Foundation

CLI + API + Orchestrator. Go-first.

- input_hash gate
- PostProcessor chain

Q3 · NOW

ep03 — Pilot

Реальные спеки одной команды.

- Telemetry · SLO
- Шаблоны под стандарты X5

Q4 · NEXT

ep04 — AI layer + Multi-lang

F3 PUML → сервисы · Kotlin, Python.

- litellm · Qwen 3 · GLM 4.7
- Web UI · CI/CD плагин

TAKE-AWAYS

Что забрать с доклада

ДЕТЕРМИНИЗМ — ФИЧА

SHA-256

Одинаковый вход = одинаковый выход. Хэш в БД подтверждает каждую генерацию.

НА ХОСТЕ — ТОЛЬКО

Docker

Ни Go, ни Java, ни Python. Тулчейн живёт внутри prod-образа.

ЭТАПЫ

2 → пилот

ep01 доказал идею CTO. ep02 дал CLI пилотной команде. ep03 — метрики на реальных спеках.

Q&A

СПРАШИВАЙ ВСЁ · КОМАНДА НА СВЯЗИ

- #aicreator в Mattermost
- petrorossolov@x5.tech
- gitlab.x5.tech/platform/aicreator